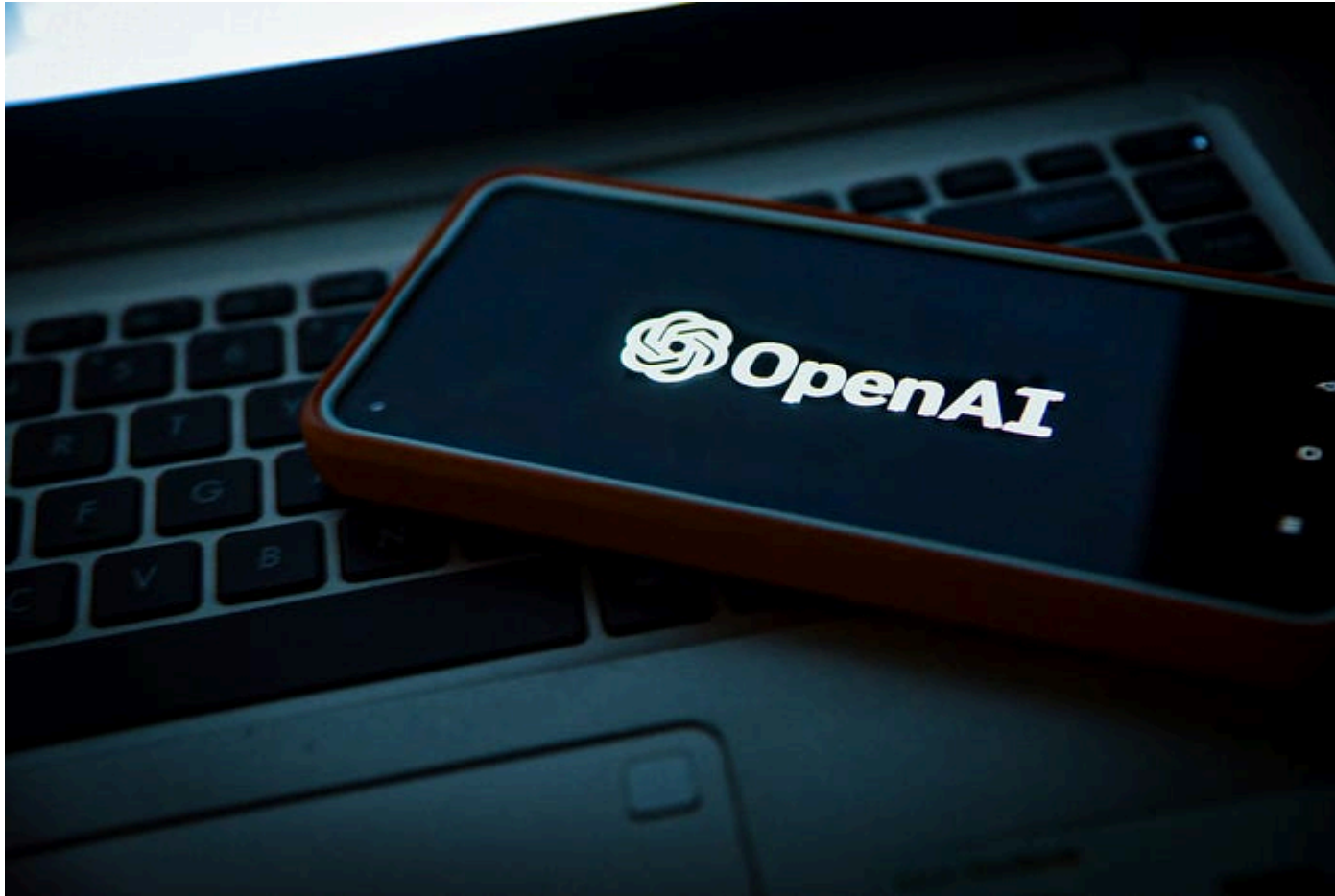


[< Go to the original](#)



RAG Without Embeddings? Here's how OpenAI is doing this...

**Gaurav Shrivastav**

Follow

∞ Level Up Coding a11y-light ~9 min read · June 11, 2025 (Updated: July 23, 2025) · Free: No

If you're building with large language models (LLMs), you've probably wrestled with Retrieval Augmented Generation (RAG). It's brilliant for grounding LLMs with external data, but the "R" part — Retrieval — can be complex.

I know I've spent hours pondering chunk sizes, overlap strategies, and which embedding model to pick. And managing vector databases? That's a whole infrastructure challenge.

So, when I stumbled upon OpenAI's recent work in their Cookbook on a different RAG approach — one that promises to bypass traditional embeddings — my developer brain did a little flip. Could we really get great retrieval results without the vector database dance? This idea felt genuinely pattern-breaking.

The secret sauce seems to be leveraging the massive context windows of newer models like GPT-4.1 and Gemini Flash. Models that can process a million tokens

Beyond the Standard RAG Pipeline

Traditional RAG is straightforward:

- **Ingest:** Split documents, embed chunks, store in a vector database.
- **Query:** Embed the query, search the DB for similar chunks.
- **Augment & Generate:** Feed chunks + query to the LLM for an answer.

This requires heavy preprocessing and managing that vector index. It's a robust approach, but it introduces latency, storage overhead, and a dependency on the quality of embeddings. Moreover, the chunking process itself can lead to a loss of context, where relevant information is split across different chunks and missed by the retrieval.

The Agentic Approach: Mimicking Your Brain

OpenAI's Agentic RAG is different. It's a multi-step process that mimics how you might tackle a long document to find an answer. You wouldn't read every word; you'd skim, find promising sections, dive deeper, and pull out the relevant bits.

embeddings," or is the LLM itself implicitly creating and using a form of "semantic representation" within its attention mechanism?

Peeking at the Code: A Multi-Step Process

Load the Document

Copy

```
import requests
# ... other imports ...
def load_document(url: str) -> str:
    """Load a document from a URL and return its text content."""
    print(f"Downloading document from {url}...")
    response = requests.get(url)
    response.raise_for_status()
    # ... (pdf reading and text extraction) ...
    return full_text
# Load the document (e.g., a legal manual with 900k+ tokens)
document_text = load_document("https://www.uspto.gov/sites/default/files/document")
```

Initial Split

```
# ... (TOKENIZER_IMPORT) ...
def split_into_20_chunks(text: str, min_tokens: int = 500) -> List[Dict[str, Any]]:
    """Split text into up to 20 chunks, respecting sentence boundaries."""
    sentences = sent_tokenize(text)
    tokenizer = tiktoken.get_encoding(TOKENIZER_NAME)
    # ... (logic to build chunks from sentences and handle chunk size) ...
    return chunks

document_chunks = split_into_20_chunks(document_text, min_tokens=500)
# This results in ~20 chunks, each potentially holding tens of thousands of tokens
```

Routing & Navigation (The Core Agent)

Copy

```
# ... (OpenAI client setup) ...
def route_chunks(question: str, chunks: List[Dict[str, Any]],
                 depth: int, scratchpad: str = "") -> Dict[str, Any]:
    """Ask the model which chunks contain info relevant to the question."""
    # ... (system prompt defining role as document navigator) ...
    user_message = f"QUESTION: {question}\n\n"
    if scratchpad:
        user_message += f"CURRENT SCRATCHPAD:\n{scratchpad}\n\n"
    user_message += "TEXT CHUNKS:\n\n"
    # ... (add chunks text with IDs to user_message) ...
    # Step 1: Ask model to record reasoning in scratchpad (required tool call)
    # ... (model call with tools and tool_choice="required") ...
    # ... (process tool call, update scratchpad) ...
    # Step 2: Ask model to select relevant chunk IDs (structured output)
```

```
# ... (extract selected_ids from JSON response) ...
return { "selected_ids": selected_ids, "scratchpad": new_scratchpad }
```

The `navigate_to_paragraphs` function then calls `route_chunks` recursively, drilling down the document hierarchy until the content is granular enough.

Generate Final Answer

Copy

```
from pydantic import BaseModel, field_validator
from typing import List, Literal

class LegalAnswer(BaseModel):
    answer: str
    citations: List[str]

@field_validator('citations')
def validate_citations(cls, citations, info):
    # ... (logic to check if citations match passed valid IDs) ...
    return citations

def generate_answer(question: str, paragraphs: List[Dict[str, Any]],
                    scratchpad: str) -> LegalAnswer:
    """Generate an answer from the retrieved paragraphs."""
    # ... (prepare context string from paragraphs with IDs) ...
    # ... (system prompt focusing on answering ONLY from provided text) ...
```

```

input=[
    {"role": "system", "content": system_prompt.format(...)},
    {"role": "user", "content": f"QUESTION: {question}\n\nSCRATCHPAD: "},
],
text_format=LegalAnswer,
temperature=0.3
)
return response.output_parsed

```

Using structured output with Pydantic and `field_validator` ensures valid citations and improves traceability.

Answer Verification (The Judge)

Copy

```

class VerificationResult(BaseModel):
    is_accurate: bool
    explanation: str
    confidence: Literal["high", "medium", "low"]

def verify_answer(question: str, answer: LegalAnswer,
                  cited_paragraphs: List[Dict[str, Any]]) -> VerificationResult:
    """Verify if the answer is grounded in the cited paragraphs."""
    # ... (prepare context string from cited paragraphs with IDs) ...
    system_prompt = """You are a fact-checker for legal information.
    Your job is to verify if the provided answer:

```

```
##### (INSTRUCTIONS FOR CONFIDENCE SCORING) #####
"""
    response = client.responses.parse(
        model="o4-mini",
        input=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": f"QUESTION: {question}\n\nANSWER TO VERIFY: {answer}"},
        ],
        text_format=VerificationResult
    )
    return response.output_parsed
```

This LLM-as-Judge step acts as a critical final check, improving the system's overall reliability.

Peeling Back the Layers: A Closer Look at OpenAI's Agentic Play

While OpenAI's take on agentic retrieval is definitely pushing the envelope, it's absolutely crucial we pump the brakes for a second and really dig into where it might stumble, and how it stacks up against good old-fashioned RAG.

The Price Tag and the Wait Time:

per query. That adds up — in computational muscle and wait time. So, the big, looming question here is, how does this financial and temporal burden *really* scale as the data gets bigger and the questions get knottier? Could we maybe shave off some of that expense? Perhaps caching some of the intermediate steps or using smaller, nimbler models just for those initial routing calls could be smart moves.

Scaling Pains:

Alright, so it feels pretty great when you're just sifting through one document. But switch gears to searching across *thousands*? That's a whole different ballgame entirely. At that kind of scale, you're probably going to *have* to pre-filter or index things at a document level; it stops being optional and starts being essential. What feels like a really promising way forward is a hybrid setup — you use something fast and rough-and-ready to quickly narrow down the possibilities, *then* unleash the agentic system for that deep-dive reasoning within the handpicked few.

The Unavoidable Context Crunch:

incredibly complex documents, this agentic system could still run into limitations. The puzzle then becomes: how gracefully does it manage documents that just blow past that limit? Maybe future iterations could get clever, perhaps using iterative summarization or smart context compression techniques to get around this hurdle.

Not All Queries Are Created Equal:

You know, not every question is the same beast. Simple, fact-based stuff? Yeah, that probably sails through relatively quickly. But those multi-part queries, or ones that are a bit more abstract? They're bound to be tougher. Does that internal "scratchpad" mechanism really provide enough support for sophisticated thinking, or do we need extra tools or reasoning aids built in?

The Ghost in the Machine: Hallucination:

Look, any system leaning on LLMs has that risk of just making stuff up — hallucination. The multi-step verification step in this agentic pipeline is designed precisely to combat that. But how bulletproof *is* it? Are there tricky edge cases where the 'judge' model might miss subtle inaccuracies? Honestly, doing a side-

Embeddings: Gone, or Just Integrated?:

Finally, a point that's a bit more philosophical but worth chewing on: is this genuinely "embedding-free"? Because deep inside its workings, specifically within that attention mechanism, the LLM is absolutely building a kind of internal representation of the input — which, arguably, is a dynamic embedding right there. This subtly shifts how we see the whole process. Instead of scrapping embeddings entirely, the agentic method seems to be weaving them directly *into* the core retrieval and reasoning loop, changing *where* and *how* they're used, rather than simply tossing them out.

Upsides and Downsides: Navigating the Choices

This agentic flavour of RAG, while undeniably clever, certainly comes with its own set of trade-offs. On the plus side, you gain that fantastic zero-ingest latency — meaning you can query brand new documents *immediately*, no tedious waiting around for preprocessing. There's a real elegance in how it moves through a document, kind of mimicking how a human might read, diving deeper where needed. This dynamic, hierarchical navigation can often dig up more spot-on

the whole indexing infrastructure headache, making setup a fair bit simpler. And critically, particularly when accuracy is paramount, that explicit, structured verification process adds a crucial layer of trust, making sure the answer it gives you is genuinely supported by the source material.

But, as always, these perks don't come free. The system naturally incurs a higher per-query cost because each question fires off multiple LLM calls instead of just a quick, cheap vector database lookup. Meaning, yeah, things can take a bit longer — you'll likely notice increased latency due to that recursive document exploration. On top of that, this specific, index-free implementation can feel a bit constrained on the scalability front compared to classic RAG when you're staring down truly colossal piles of documents, unless you layer in some initial pre-filtering step.

Knowing When to Reach for It

So, considering this give-and-take, where does this approach really hit its stride? It seems particularly well-suited for tackling complex questions within lengthy, specialized documents — think detailed legal briefs or dense technical manuals — where getting the facts *exactly* right, and being able to point to them with

you have to keep rebuilding. On the flip side, if your need is for a lightning-fast, general chatbot that needs to pull information from vast, static datasets in a flash, traditional embedding-based RAG might still be your champion.

Kicking Around Future Concepts

The minds at OpenAI are naturally thinking about how this could evolve. One thought is a hybrid system: maybe use traditional embedding search to quickly narrow down the field, *then* let the agentic system roam and reason within that smaller, more manageable pool. Another really cool possibility? Leveraging those massive context windows to build a detailed knowledge graph from a document just once, as a sort of upfront task, and then letting a different, perhaps lighter, model zip through *that* graph for answers. Kind of marrying the strengths of both worlds, wouldn't you say? And finally, dialing in the 'granularity' — letting users easily specify how deep they want the system to go, needing sentence-level proof (critical for legal work) versus maybe just a paragraph (which might be totally fine for sifting through news articles). It's all about finding that sweet spot between pinpoint accuracy and operational cost.

Pulling It All Together

kind of fluid, on-the-fly document exploration. The way it reads, moving through the hierarchy, feels natural and absolutely helps nail down relevance on trickier questions. Having that scratchpad? It genuinely bumps up the reasoning by giving the model space to work things out step-by-step. And the fact you can get started pretty quickly without needing a whole database setup (at least for the core retrieval bit) is a definite win depending on what you're trying to achieve. But crucially, crucially, that built-in check makes you trust the answers more — knowing they're actually pulled directly from and verified against the source.

Overall? This Agentic RAG thing is genuinely exciting. It really shows just how LLMs, particularly those with big context windows and agentic chops, can crack open complex information problems in fresh, powerful ways — especially when getting it *right* and understanding deeply matters more than being lightning-fast or handling sheer, brute-force volume.

Here is the [Colab Notebook](#) for the full code.

Thanks for reading, and as always, I'll see you in the next one!

[#openai](#) [#embedding](#) [#agentic-ai](#) [#ai-agent](#) [#agents](#)

Freedium

[ko-fi](#) [librepat](#) [patreon](#)
